Message Intelligence System

Theory Concepts Document

Probability & Classification Theory behind the Spam / Legitimate Message Classifier

Theory concepts used in:

Conditional Probability · Bayes' Theorem · Naive Bayes · KNN · SVM · Evaluation Metrics

Submitted by: **Devanshi Bachhote** · GR ID: **10775**

Course: **AI ML and Data Science**

1. Introduction and Project Overview

The goal of the project is to build a **Message Intelligence System**: given an incoming digital message, the system must decide whether it is **legitimate (label 0)** or **spam (label 1)**. This is a **binary classification** problem — a supervised learning task where each training example already carries the correct answer and the model must learn a rule that generalises to unseen messages.

We work with a dataset of **5200 messages**, described by **12 numeric features** extracted from the text and from sender behaviour (message length, word count, number of URLs, digit count, special characters, spam/legit keyword scores, sender activity score and account age, messages sent in the last 24 hours, hour of day, day of week). The target is **spam_label**. The classes are imbalanced: **81.3% legitimate vs 18.7% spam** — an important detail when we later judge the models with evaluation metrics. The data is split 80/20 (4160 train / 1040 test) with stratification, and distance/margin-based models are trained on **standardised** features.

The project compares three classifier families — **distance-based (KNN)**, **margin-based (SVM)** and **probabilistic (Naive Bayes)** — and this document explains the theory they are built on, why each concept matters, and how it showed up in our experiments.

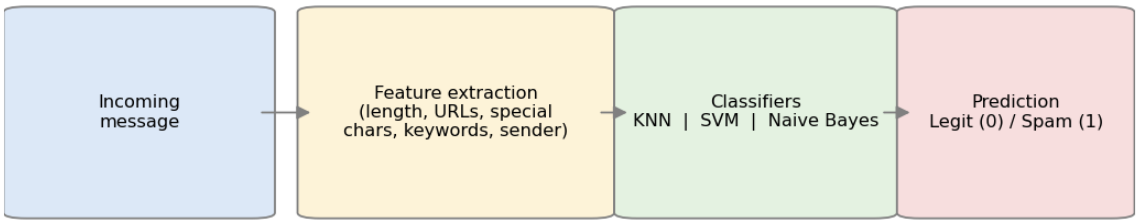


Fig 1. The system pipeline: raw message → numeric features → classifiers → spam/legit decision.

2. Probability Foundations

2.1 Basic probability

Definition — Probability.

The probability of an event A is a number between 0 and 1 measuring how likely A is to occur: P(A) = (favourable outcomes) / (total outcomes). The set of all possible outcomes is the **sample space**.

In our project the “experiment” is: pick a random training message and look at its label. Out of 4160 training messages, 778 are spam, so our before-any-evidence belief is **P(spam) = 778/4160 = 0.187**. This number is called the **prior** probability.

2.2 Conditional probability

Definition — Conditional Probability.

The conditional probability of A **given** B is the probability of A when we already know B happened: the condition shrinks the sample space to only the cases where B is true.

$$P(A \mid B) = P(A \cap B) / P(B), \quad \text{with } P(B) > 0$$

Project example (computed by counting rows): 849 of the 4160 training messages contain at least one URL, and 579 of those are spam. Therefore:

$$P(\text{spam} \mid \text{has URL}) \;=\; 579 / 849 \;=\; \mathbf{0.682}$$

Analysis from our project. Knowing the message has a URL moves our belief from 0.187 to 0.682 — almost 4× more suspicious. Other conditions behave the same way: P(spam | message longer than 80 chars) = 0.293 (weak evidence), while P(spam | 2 or more special chars) = 1.000 — every such training message was spam. A classification algorithm is, at heart, a machine for estimating P(class | features) like this — but for *all features together* instead of one.

3. Bayes' Theorem

Conditional probability P(A|B) is easy to count, but a classifier needs the *reverse* direction — P(class | evidence). Bayes' theorem (Thomas Bayes, 18th century) does exactly this reversal:

$$P(A \mid B) \;=\; P(B \mid A) \cdot P(A) / P(B)$$

Term	Symbol	Meaning	In this project (real values)
Prior	P(A)	Belief about class A before seeing evidence	P(spam) = 778/4160 = 0.187
Likelihood	P(B | A)	How typical the evidence is inside class A	P(URL | spam) = 579/778 = 0.744
Evidence	P(B)	Overall rate of the evidence	P(URL) = 849/4160 = 0.204
Posterior	P(A | B)	Updated belief after seeing evidence	P(spam | URL) = 0.682

3.1 Verifying Bayes' theorem on our data

We computed P(spam | URL) two independent ways — direct counting, and Bayes' formula — and got the same value:

$$P(\text{spam} \mid \text{URL}) = P(\text{URL} \mid \text{spam}) \cdot P(\text{spam}) / P(\text{URL})$$
$$= (0.744 \times 0.187) / 0.204 = \mathbf{0.682} \quad \text{(direct count: 0.682 ✓)}$$

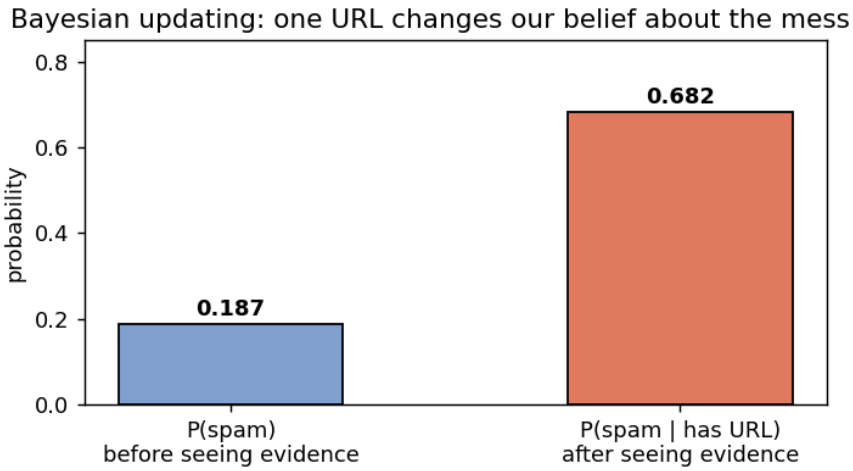


Fig 2. Bayesian updating: evidence "message has a URL" raises the spam belief from 0.187 to 0.682.

3.2 Why Bayes' theorem matters for classification

A classifier ultimately wants the **posterior** P(class | features). Training data, however, tells us the **likelihood** direction more easily (e.g. "among spam messages, how often do we see 3 URLs?"). Bayes' theorem is the bridge between the two. The optimal decision rule derived from it is the **MAP rule** — pick the class with the largest posterior:

$$\hat{y} = \arg\max_c \; P(c \mid x) \;=\; \arg\max_c \; P(x \mid c) \cdot P(c)$$

The denominator P(x) is dropped because it is identical for all classes and does not change the arg max. This rule is the theoretical heart of the Naive Bayes classifier used in Part E of the project.

4. The Naive Bayes Classifier

4.1 The "naive" independence assumption

Assumptions of Naive Bayes.

1. All features are conditionally **independent given the class** (the naive part). **2.** All features are equally important. **3.** (Gaussian NB only) each numeric feature follows a **normal distribution** within each class.

Thanks to assumption 1, the joint likelihood factorises into a simple product of per-feature probabilities — this is what makes NB trainable with a handful of statistics (mean and variance per feature per class):

$$P(c \mid x_1, \ldots, x_n) \;\propto\; P(c) \cdot \prod_i P(x_i \mid c)$$

For Gaussian NB each term is a normal (bell-curve) likelihood:

$$P(x_i \mid c) = 1 \,/\, \sqrt{(2\pi\, \sigma_{i,c}^2)} \cdot \exp\!\left(-(x_i - \mu_{i,c})^2 \,/\, (2\sigma_{i,c}^2) \right)$$

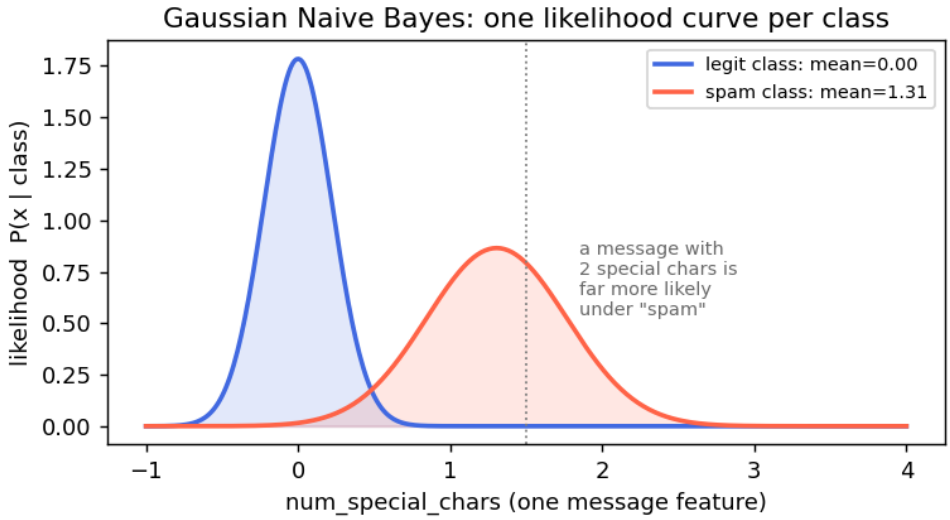


Fig 3. Per-class Gaussian likelihoods of the feature num_special_chars, learned from training data.

4.2 Hand computation matching the model

In the notebook we recomputed the posterior for three test messages **by hand**, using only the per-class means, variances and priors (log-space to avoid numerical overflow). The results matched sklearn's *predict_proba* exactly — e.g. P(legit)=1.0000, P(spam)=0.0000 for a normal business message. This confirms the model is literally executing the Bayes calculation of Section 3.

4.3 Impact of the assumptions (analysis)

Analysis from our project. The independence assumption is **false** for our features (message_length and word_count are correlated, and the keyword/URL/digit features move together). Consequences we actually observed: (a) **over-confident probabilities** — correlated features are counted as independent evidence, so posteriors collapse to 0.0000/1.0000; (b) despite that, naive Bayes reached accuracy **0.999**, precision **0.995**, recall **1.000** — because classification only needs the arg max of the posterior, which is robust even when the probability values are not calibrated; (c) its single mistake was a false positive on a genuine message containing a URL — exactly the kind of borderline case where violated independence assumptions hurt most.

NB also needed **no feature scaling** (each feature is modelled separately), trained in a few milliseconds, and its model is just 2 × 12 means, 12 variances and the two priors — trivial to store and inspect.

5. K-Nearest Neighbors (Distance-Based Learning)

5.1 Working principle

KNN in one line.

A message gets the majority label of the K training messages most similar (closest) to it in feature space — "tell me who your neighbours are and I will tell you what you are".

Steps: (1) store all training points; (2) for a new message compute its distance to every stored point; (3) sort distances and take the K smallest; (4) output the majority class of those K neighbours. KNN is a **lazy learner**: all the work happens at prediction time.

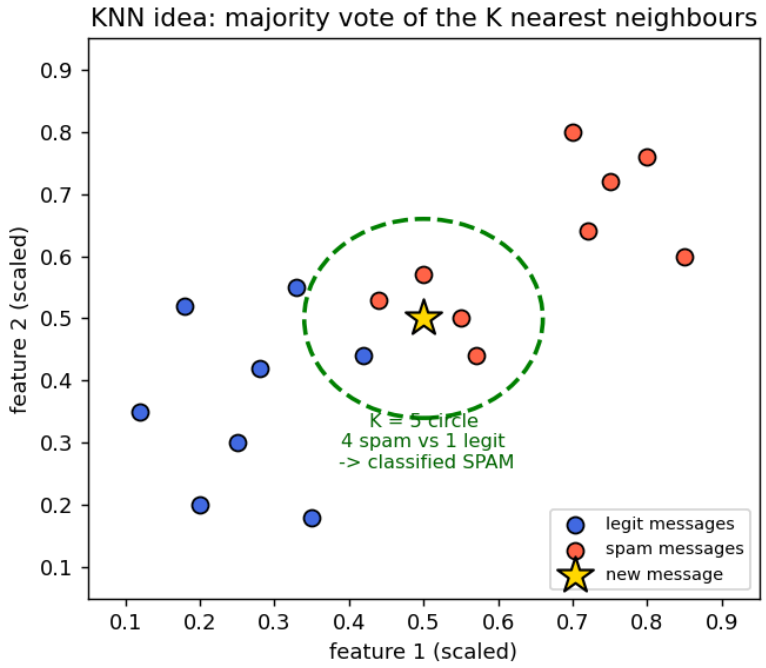


Fig 4. KNN sketch: the K=5 circle contains 4 spam and 1 legit message, so the new message is classified spam.

5.2 Distance metrics

$$\text{Euclidean:}\quad d(x, z) = \sqrt{\left(\Sigma_i\,(x_i - z_i)^2\right)}$$
$$\text{Manhattan:}\quad d(x, z) = \Sigma_i\,|\,x_i - z_i\,|$$

Euclidean distance is the straight-line distance and is the default; Manhattan distance sums the per-feature differences ("walking on a grid") and is slightly more robust to a single feature with a big value. In our scaled experiments all metrics performed almost identically (≈100%), because the classes are well separated — but on **unscaled** features the metric choice clearly mattered (Euclidean 0.968, Manhattan 0.977, Chebyshev 0.959, Cosine 0.976).

5.3 Why scaling is essential (analysis)

Analysis from our project. KNN on raw, unscaled features scored only **0.968** accuracy with **33 mistakes** (24 spam missed, 9 legit flagged). After standardisation it reached **1.000**. Reason: distances are sums of per-feature differences, so a feature with range ≈1000 (sender_account_age_days) completely drowns out features with range ≈4 (num_urls). StandardScaler (mean 0, std 1, fitted on training data only) makes every feature contribute fairly.

5.4 Choosing K and failure cases (analysis)

Varying K from 1 to 25: K≤5 gave 100% accuracy, while K≥7 dropped slightly to 99.9% — larger neighbourhoods are smoother and start including points from the other class near the boundary (underfitting), whereas tiny K can memorise noise (overfitting). Even at its best, KNN missed borderline spam such as *"You are eligible for a refund of a brand new phone today only!"* — a message whose (URL-free, clean-looking) feature vector sits among legit messages. Distance-based models fail exactly on such "camouflaged" points.

6. Support Vector Machine (Margin-Based Learning)

6.1 Hyperplane, margin and support vectors

SVM in one line.

Find the hyperplane that separates the classes with the **widest possible margin**; only the points on the margin — the **support vectors** — determine the boundary.

Boundary: $w^T x + b = 0$ · Decision: $\text{sign}(w^T x + b)$
Margin distance from boundary: $1/||w||$ · Total width: $2/||w||$

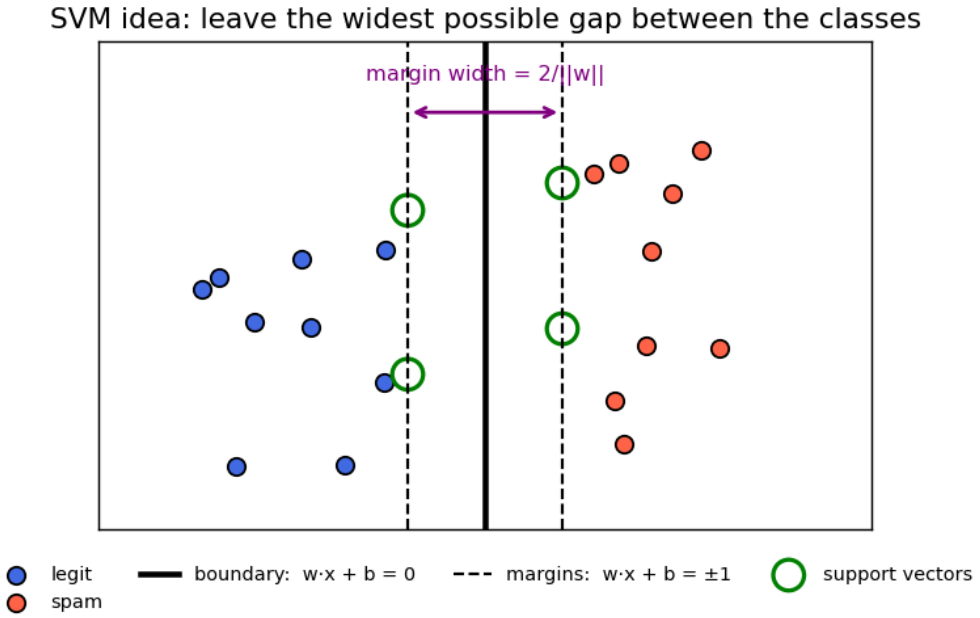


Fig 5. The SVM boundary (solid), margins (dashed) and support vectors (green rings).

Maximising $1/||w||$ is a well-posed optimisation problem (quadratic programming). Because only support vectors matter, SVM is memory-efficient and robust: moving or adding points far from the boundary changes nothing.

6.2 Kernel trick

When classes cannot be separated by a straight line, a **kernel** implicitly lifts the data into a higher dimension where a flat hyperplane corresponds to a curved boundary in the original space — without ever computing the lifted coordinates explicitly:

Kernel	Formula	Good for
Linear	$K(x, z) = x^T z$	Separable or high-dimensional data; fast
Polynomial	$K(x, z) = (x^T z + 1)^d$	Interactions between features
RBF (Gaussian)	$K(x, z) = \exp(-\gamma \, \|\|x-z\|\|^2)$	Smooth curved boundaries; general default

6.3 Margin behaviour in our project (analysis)

Analysis from our project. The classes in our dataset are nearly linearly separable, so all kernels reached accuracy 1.000 — but the margins tell a richer story. The linear kernel needed only **10 support vectors** (5 per class) out of 4160 training messages, meaning the entire decision rule is summarised by 10 points. The RBF kernel used 138 support vectors — it bends the boundary more than necessary here. Using just the two most informative features, the linear SVM still achieved 94.4% with a margin $1/||w|| \approx 0.73$ (in scaled units), which is the plot in the notebook. Among the tested models the linear SVM was also the **fastest at prediction**, which is why it was chosen for deployment in the final report.

7. Comparing the Three Classifier Families

Each family represents a different philosophy of learning. The table summarises the theory-level differences:

Aspect	Distance-based (KNN)	Probabilistic (Naive Bayes)	Margin-based (SVM)
Core idea	Majority vote of K closest points	Bayes posterior per message	Widest-margin hyperplane
Model learned?	None (stores data)	Means, variances, priors	Weights of hyperplane
Basis of decision	Geometry / similarity	Probability theory	Optimisation / geometry
Needs scaling	Yes — critical	No	Yes — recommended
Training cost	≈ 0	Very low	Low (linear) – medium (RBF)
Prediction cost	High (compares with all)	Very low	Very low
Output	Label only	Label + probability	Label (+ margin distance)
Best when	Local patterns, small data	Many features, fast baselines	Clear separation, robustness
Main weakness	Slow at scale, scaling-sensitive	Broken independence → overconfident	Tuning C/kernel can be tricky

8. Evaluation Metrics (and Why Accuracy Alone Lies)

All models are judged on the 1040-message test set using the **confusion matrix**: TP (spam caught), TN (legit kept), FP (legit wrongly flagged), FN (spam missed):

	Predicted: Legit	Predicted: Spam
Actual: Legit	TN — correctly kept	FP — false alarm (annoying for users!)
Actual: Spam	FN — scam slipped through	TP — spam correctly caught

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$
$$\text{Precision} = TP / (TP + FP) \quad \cdot \quad \text{Recall} = TP / (TP + FN)$$
$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

Why not accuracy alone? With 81% legit messages, a lazy model that labels *everything* legit already scores 81% accuracy while catching zero spam. Precision answers *"when I say spam, how often am I right?"* and recall answers *"of all real spam, how much did I catch?"*. A spam filter usually wants high precision (never lose a client invoice) while a security scanner wants high

recall (never let a scam through) — the two trade off, and F1 balances them.

8.1 Results obtained in the project

Model	Accuracy	Precision	Recall	F1	Note
KNN (k=5, scaled)	1.000	1.000	1.000	1.000	1 error if K too big; slow prediction
KNN (k=5, unscaled)	0.968	0.950	0.877	0.912	33 errors — shows why scaling matters
SVM linear	1.000	1.000	1.000	1.000	only 10 support vectors; fastest prediction
SVM RBF	1.000	1.000	1.000	1.000	138 support vectors; no gain over linear here
Naive Bayes	0.999	0.995	1.000	0.997	1 false positive; probabilities over-confident

Analysis from our project. All scaled models tie at 1.000 — the dataset separates cleanly once features are standardised. Naive Bayes keeps perfect recall (no spam missed) but pays one false alarm (precision 0.995). For **high precision** choose KNN/SVM (zero false alarms here); for **high recall** Naive Bayes was the safest. The final business choice was the linear SVM (accuracy + speed), with NB as a high-recall safety net quarantining suspicious messages instead of deleting them.

9. How Theory Explained Our Results

Theory concept	Where it appeared in the project
Conditional probability	Counted P(spam | URL) = 0.682 vs prior 0.187 — evidence dramatically changes belief
Bayes' theorem	Verified numerically (0.682 both ways); MAP rule = what every NB prediction does
Naive independence	Hand-computed posteriors matched the model; over-confidence 0.0000/1.0000 explained by correlated features
Gaussian likelihood	Per-class mean/variance curves (Fig 3) decide a feature's vote for each class
Distance geometry	Unscaled KNN 0.968 → scaled 1.000; camouflaged spam near the border misclassified
Margin maximisation	Linear SVM generalises with only 10 support vectors; 94% using just 2 features
Confusion-matrix metrics	81/19 imbalance means accuracy alone would hide a useless model; precision/recall revealed NB's trade-off

Assumptions we made: message_id, timestamp and raw text were excluded from modelling (identifiers / free text); missing values (<4% per column) were filled with the median; an 80/20 stratified split kept the spam ratio fixed; scaling was applied where the algorithm geometry requires it.

9.1 Conclusion

The three classifier families solved the same problem through three different lenses — similarity (KNN), separation (SVM) and probability (Naive Bayes). The probability concepts are not

decoration: conditional probability explains why evidence like URLs changes suspicion, Bayes' theorem is the exact computation NB performs, and understanding its independence assumption explains both NB's success (perfect recall) and its failure mode (over-confident probabilities). Choosing a model in practice means weighing accuracy against speed, interpretability, and which errors the business can tolerate.

References

1. James, Witten, Hastie, Tibshirani — *An Introduction to Statistical Learning* (chapters on classification, KNN, SVM, naive Bayes).
2. C. Bishop — *Pattern Recognition and Machine Learning* (probability theory, chapter 1).
3. scikit-learn documentation — user guide sections on neighbors, svm and naive_bayes.
4. Course lecture notes on probability and supervised learning.